



Srinath Reddy - Founder & CEO



BigHammer.ai

Moving Spark jobs to EMR or Dataproc? **3 things break quietly** that your tests won't catch.

Plus the 5 checks I'd run before switching anything off.

same query · two engines

```
SELECT CAST('abc' AS INT) AS amount;
```

```
-- Databricks Runtime 15.4 cluster (ANSI off)
```

```
→ NULL
```

```
-- Spark 4.0 on EMR 8.0 or Dataproc 3.0
```

```
→ [CAST_INVALID_INPUT] job fails
```

3 quiet breakers, 5 checks that catch them.



Srinath Reddy - Founder & CEO



BigHammer.ai

Why now. The target platforms moved to Spark 4.0.

- **EMR 8.0** shipped Spark 4.0 in June 2026.
- **Dataproc image 3.0** went GA in July 2026.
- Spark 4.0 turns **ANSI mode** on by default.
- Databricks Runtime only did the same from **17.0**.
Most older jobs ran with it off.

DBR before 17

ANSI off on clusters.
Bad casts become
NULL.

DBR 17 / Spark 4.0

ANSI on by default. Bad
casts throw.

EMR 8 · DATAPROC 3

Spark 4.0 is where your
jobs land.

So a job that "worked" for years can fail, or quietly change, on day one.

Same code. New defaults.



1. ANSI mode throws where Spark used to shrug.

Bad input used to become NULL, or silently wrap. Now it stops the job.

spark 4.0

```
SELECT 2147483647 + 1;
-- ANSI off: -2147483648 (silent wrap)
-- Spark 4.0: [ARITHMETIC_OVERFLOW]

SELECT to_date('15/10/2026', 'yyyy-MM-dd');
-- ANSI off: NULL
-- Spark 4.0: [CANNOT_PARSE_TIMESTAMP]
```

FIX Use `try_cast` and `try_add` where NULL was the intent, and set `spark.sql.ansi.enabled` explicitly per job.

NULL used to hide bad data. Now it fails loudly.



2. Time zones shift your dates.

Timestamps default to `TIMESTAMP_LTZ`, read in the **session time zone**.

one event, two sessions

```
event_ts = 2026-10-15 02:30 UTC
```

```
-- session time zone UTC
```

```
to_date(event_ts) → 2026-10-15
```

```
-- session time zone America/New_York
```

```
to_date(event_ts) → 2026-10-14
```

Same row, different day. Daily totals move and nobody gets an error.

FIX Pin `spark.sql.session.timeZone` in every job, and compare one day of output across both engines.



3. Some code only exists on Databricks.

● ● ● grep before you migrate

```
dbutils.widgets.get("run_date") # notebook only
dbutils.fs.ls("/mnt/raw")        # DBFS
import dlt                       # old module
spark.table("main.sales.orders") # 3-part names
```

DATABRICKS CALL	PORTABLE REPLACEMENT
<code>dbutils.widgets</code>	job parameters
<code>dbutils.fs</code>	plain cloud paths (s3://, gs://)
<code>import dlt</code>	plain jobs (pipelines need Spark 4.1+)
<code>catalog.schema.table</code>	your target catalog's names

NOTE Photon is different: it's an engine swap, not a code break. Expect speed changes, not failures.



Srinath Reddy - Founder & CEO



BigHammer.ai

The **parity checklist.** Run it on every migrated job.

1

Row and column counts

The fastest signal something is off

2

Schema

Same columns, same types

3

Null rates per column

Catches silent handling changes

4

Numeric aggregates

Sums, mins and maxes on key measures

5

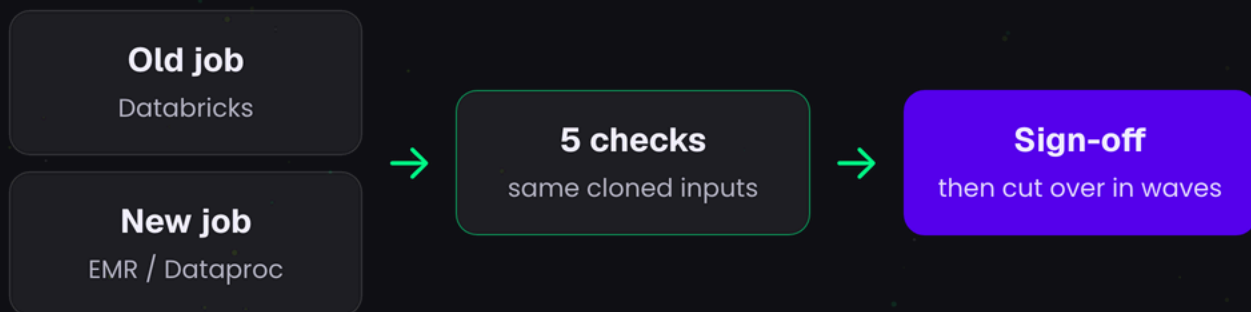
Row-level compare

By business key, or by hash if there's no clean key

Agree tolerances up front. Keep the evidence after.



How to run it without touching production.



- Run old and new side by side on **cloned data**.
- Agree the checks and tolerances with the **business owner first**.
- Cut over in **small groups** of jobs, never all at once.
- Keep the comparison results. That's your **sign-off evidence**.

Conversion is visible. Parity is where trust is won.



Srinath Reddy - Founder & CEO



BigHammer.ai

Want the full playbook? Assess, migrate, monitor.

I'm walking through it live, with real numbers, then taking your questions.



Reduce your Databricks costs by up to 75%

Thu 15 Oct · 12:00 PM ET · 5 PM UK

webinar.bighammerai.com